Product Requirements Document

Personal Watermark Web App (Mobile-First)

Author	Ahsan (Product Owner) — drafted with Claude, Senior Dev advisory role
Status	Draft v1.0 — ready for implementation handoff
Date	July 21, 2026
Reference	watermarkktp.com (existing open-source tool, MIT licensed)
Deployment Target	GitHub Pages (free static hosting)

1. Overview

This is a personal-use, single-page web application that lets a user apply a watermark (text and/or logo image) to one or more images entirely in the browser. No image, file, or personal data is ever uploaded to a server — all processing happens client-side using the HTML5 Canvas API, consistent with the privacy model of the reference tool (watermarkktp.com), which the community independently verified as fully client-side by testing it offline.

The primary improvement over the reference tool is a mobile-first, touch-friendly redesign, while remaining fully responsive on desktop. Scope also extends past the original feature set with logo watermarking, free-drag positioning, and batch (multi-image) processing.

2. Goals

- Recreate the core value of the reference tool: fast, private, in-browser image watermarking.
- Make the experience genuinely mobile-first — usable comfortably with one thumb on a phone, not just "responsive" as an afterthought.
- Add three specific capabilities beyond the original: image/logo watermark, free-drag watermark positioning, and batch processing of multiple images.
- Ship as a static site with zero backend, deployable directly to GitHub Pages Free tier.

2.1 Non-Goals (explicitly out of scope for v1)

- No user accounts, login, or cloud storage of any kind.
- No server-side processing, APIs, or databases.
- No direct camera capture flow — image input is via file picker / drag-drop only (gallery-based, not live camera).
- No saved presets / persisted settings in browser storage (localStorage) — every session starts clean, reinforcing the "nothing is retained" privacy story.
- No installable PWA / offline app shell in v1.
- No auto-generated watermark text (e.g. auto date-stamping) — out of scope unless requested later; user types their own text.

3. Target User

Primary user: the product owner (Ahsan), for personal use watermarking scanned ID documents and similar images before sharing them. Single-user context — no multi-tenant or collaboration requirements.

4. Technical Approach

4.1 Stack decision

Plain HTML, CSS, and vanilla JavaScript. No framework (React/Vue) and no build step.

Rationale:

- GitHub Pages Free serves static files directly — no build pipeline, no CI step, no dependency drift to maintain over time.
- The app's actual complexity is a Canvas-based image transform plus a form — this does not need component-framework overhead.
- Easier to hand off as a single implementation task with no scaffolding/tooling decisions left ambiguous.
- Trade-off acknowledged: state management (multiple images, undo of settings) is more manual than it would be in React. Given the scope (single user, session-only state, no persistence), this is acceptable.

4.2 Core technology

- Rendering/compositing: HTML5 Canvas API.
- Image input: File API (drag-and-drop + <input type="file"> picker), multiple file selection for batch mode.
- Output: Canvas.toBlob() / toDataURL() triggering a client-side download — no network call involved.
- Hosting: GitHub Pages, static files only.

5. Functional Requirements

5.1 Image Input

- Drag-and-drop area for image upload.
- Tap-to-browse fallback for mobile (gallery picker) — this is what stands in for camera capture, since direct camera access is out of scope.
- Accept multiple images at once (required for batch mode).
- Show thumbnail previews for all loaded images; user can select which one is active in the editor.

5.2 Watermark Content

- Text watermark: free-text input, font selection (minimum: the 3 fonts from the reference — Times New Roman, Arial, PT Sans), color picker, size control, transparency/opacity slider, rotation control.
- Image/logo watermark (new): user uploads a PNG/JPG (ideally transparent PNG) to use as the watermark instead of, or alongside, text.
- User can toggle between text mode, logo mode, or both simultaneously.

5.3 Positioning (new: upgraded from the original)

- 9-position preset grid (top/center/bottom × left/center/right) — kept from the original as a quick-select shortcut.
- Free-drag positioning (new): user can drag the watermark anywhere on the canvas preview, with touch-drag support (not just mouse) as this is mobile-first.
- Preset grid taps snap the watermark to that position; dragging afterward overrides the preset.

5.4 Batch Processing (new)

- Once watermark settings (text/logo, position, size, color, rotation, transparency) are configured on one image, the user can apply the same settings across all loaded images in one action.
- Batch mode allows per-image override before export if the user wants to nudge one image's position individually (nice-to-have, not a hard requirement for v1 — flag as stretch goal).
- Output: download all watermarked images (individually, or zipped if feasible client-side — flag as stretch goal, since a zip library adds a dependency).

5.5 Controls

- Reset button to clear all watermark settings back to defaults.
- Live preview updates as any control changes (no separate "apply" step needed until export).

- Download button exports the current canvas state as an image file (PNG recommended to preserve quality/transparency).

6. Non-Functional Requirements

6.1 Privacy

- Zero network calls involving user image data, at any point in the flow — this must be independently verifiable (e.g. user can disconnect internet mid-session and the app keeps working, as was demonstrated with the reference tool).
- No analytics, tracking, or telemetry that transmits image content or metadata.
- No use of localStorage/sessionStorage for image data or presets in v1 (see Non-Goals).

6.2 Mobile-First UI/UX (the key improvement area)

- Design and build the layout for small viewports first, then progressively enhance for tablet/desktop — not the reverse.
- Touch targets sized appropriately for thumb use (no cramped desktop-style sliders/buttons ported as-is).
- Controls panel should not force excessive scrolling to reach Download — consider a collapsible/bottom-sheet style control panel on mobile, full side panel on desktop.
- Watermark drag interaction must work smoothly with touch events (touchstart/touchmove/touchend), not only mouse events.
- Responsive breakpoints: mobile (default), tablet, desktop — verified at common widths (~375px, ~768px, ~1280px).

6.3 Performance

- Canvas operations should stay responsive on mid-range mobile devices; avoid unnecessary full-canvas redraws on every drag frame if performance testing shows lag.
- Batch processing of several images should not freeze the UI thread for an unreasonable time — acceptable to process sequentially with visible progress.

6.4 Browser Support

- Modern evergreen browsers (Chrome, Safari, Firefox, Edge — mobile and desktop). No IE11 support required.

7. Risks & Open Items to Confirm During Build

Item	Note
Zip download for batch export	Adds a client-side zip library (e.g. JSZip) as a dependency — decide if that's acceptable given the "no build step" preference, or if per-image downloads are fine for v1.
Per-image override in batch mode	Flagged as stretch goal — confirm if v1 needs it or if uniform settings across all images is acceptable.
Font loading	Reference tool uses Times New Roman, Arial, PT Sans. PT Sans isn't a system font — will need to be loaded via @font-face/Google Fonts CDN link, which is a network request (fonts only, not image data) — confirm this is acceptable under the privacy goal.
Large image handling	Very large source images (e.g. 20MP+ phone camera photos) may need client-side downscaling for canvas performance — confirm if this matters for your use case.

8. Success Criteria (v1 done when...)

- User can load one or more images via drag-drop or file picker on both mobile and desktop.
- User can add a text watermark and/or logo watermark, positioned via preset grid or free drag, with adjustable size/color/rotation/transparency.
- Settings apply across all loaded images in batch, and each can be downloaded.
- The app works with the network disconnected after initial page load, proving no data leaves the device.
- Layout and controls are comfortably usable one-handed on a phone screen, not just "shrunk desktop UI."
- Deployed and reachable via GitHub Pages.

9. Handoff Note

This PRD is written to be handed to a separate Claude session (or any developer) for implementation. It intentionally specifies behavior and constraints, not exact code structure — implementation details (file layout, specific CSS approach, canvas redraw strategy) are left to the builder's discretion within the constraints above.